Islamic University of Technology

Board Bazar, Gazipur, Dhaka

SWE 4304 – Software Project Lab-I

Final Project Report

SMART GROCERY CLI

Grocery Shop Management System

SPL TEAM-10

SPL Team - 10

Team Members:

Ayman Rahman Bhuiyan
ID: 230042140
BSc. in Software Engineering Department of Computer Science and Engineering

Arafat Hosen
ID: 230042151
BSc. in Software Engineering Department of Computer Science and Engineering

Md. Jahirul Islam Shifat
ID: 230042156
BSc. in Software Engineering Department of Computer Science and Engineering

Supervisor:

Mueeze Al Mushabbir
Lecturer
Department of Computer Science & Engineering
Islamic University of Technology

1. Problem Statement

Small grocery store owners in many local communities still depend heavily on manual stock management. This manual approach leads to several operational challenges.

Problems faced by shop owners:

1. Inventory Mismanagement
Manual stock tracking often leads to miscalculations, inaccurate inventory records, and difficulty identifying low-stock products.

2. Lack of Purchase Tracking
Many small grocery shops do not maintain digital records of customer purchases, making it difficult to track transaction history or analyze customer behavior.

3. Inefficient Billing Process
Manual billing systems increase the chances of human error and slow down the checkout process.

4. No Customer Insights
Shop owners cannot identify frequently purchased products or customer spending patterns due to lack of analytics.

5. No Product Recommendation
Customers do not receive suggestions based on their previous purchases or buying preferences.

6. Dependency on Internet-based Systems
Many existing systems require internet connectivity, which is not always reliable for small local grocery stores.

These issues create inefficiencies in daily operations and reduce the ability of shop owners to make data-driven decisions.

2. Potential Solution

To address the above problems, we developed Smart Grocery CLI, an offline command-line based grocery management system.
The system digitizes grocery store operations while maintaining simplicity and offline accessibility.

The solution provides:

Inventory Management System
Allows administrators to add, update, delete, and monitor grocery products with real-time stock tracking.

User Authentication System
Supports secure login and registration with role-based access control for administrators and customers.

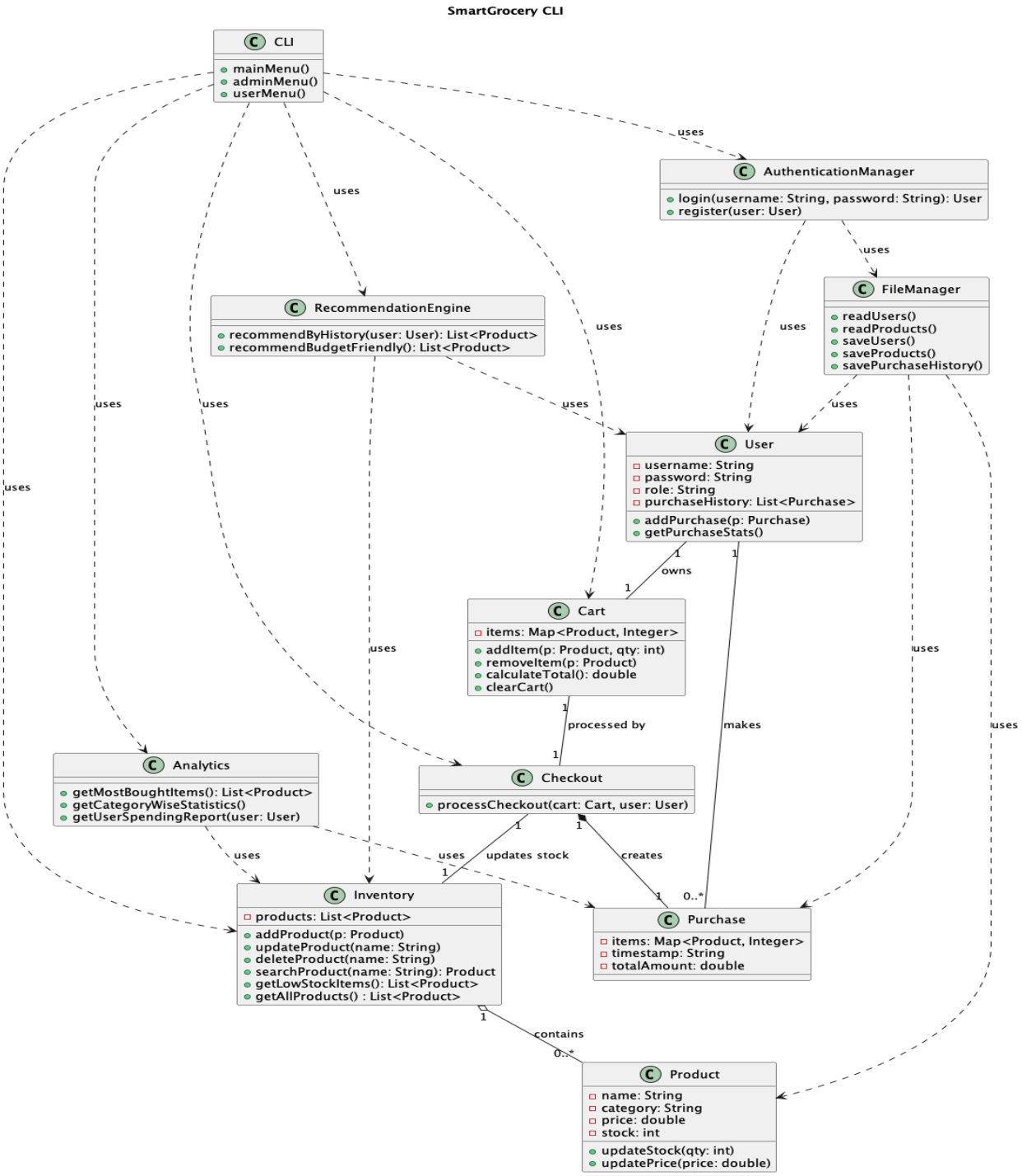
Cart and Checkout System
Enables customers to add items to a cart and complete purchases with automatic stock updates and bill generation.

Recommendation Engine
Suggests products based on purchase history, category similarity, and price comparison.

Analytics and Reports
Provides useful insights such as frequently purchased products, category statistics, and user spending summaries.

Offline File-Based Storage
All system data is stored locally using structured text files without requiring an internet connection.

3. UML DIAGRAM



UML Class Diagram Overview

The UML class diagram of **SmartGrocery CLI** illustrates the structural design of the system and the relationships among its core components. The system is designed following Object-Oriented principles to ensure modularity, low coupling, and clear responsibility separation.

The **Product** and **Inventory** classes handle grocery items and stock management, where a single inventory contains multiple products. The **User** class represents system users (Admin and Customer) and maintains purchase history, while each user owns a **Cart** to manage selected items before checkout.

The **Checkout** class processes purchases by validating cart items, updating inventory stock, and creating **Purchase** records. Each completed checkout results in one purchase entry, which is stored for future reference and analysis.

User authentication and registration are handled by the **AuthenticationManager**, separating security logic from the user interface. Persistent data storage is managed through the **FileManager**, which reads and writes user, product, and purchase information using local text files.

Additional functionality is provided by the **RecommendationEngine** and **Analytics** classes, which generate offline product recommendations and system reports based on purchase and inventory data.

The **CLI** acts as the presentation layer, providing role-based menus and handling user interaction. It depends on core system classes to execute operations but remains independent of business logic, ensuring a clean layered architecture.

Note on Primary and Foreign Keys

Primary keys and foreign keys are not explicitly identified in the UML class diagram because the SmartGrocery CLI system does not use a relational database. Instead, the system operates using object-oriented in-memory objects with file-based storage. Relationships between entities are maintained through object references rather than database keys.

4. Implemented Features

The **SmartGrocery CLI** system integrates offline inventory management, user-based purchasing, and intelligent analytics to support efficient grocery management without relying on internet connectivity or external databases. The system is designed as a headless, command-line-based application that emphasizes strong Object-Oriented Design, modularity, and data integrity through file-based storage.

The following sections describe the major functionalities implemented in the system.

4.1 User Management

The user authentication module ensures secure access to the system by verifying user credentials and managing role-based permissions. Authentication is handled through a username–password mechanism, and all user data is stored locally in text files, ensuring complete offline functionality.

● Username and Password Authentication

- Users can register and log in using valid credentials.
- Authentication verifies user identity before granting access to the system.
- User information is stored securely in local files.

● Role-Based Access Control

- The system supports two user roles:
    - **Admin**: Responsible for managing inventory and system data.
    - **Customer**: Allowed to browse products, make purchases, and view personal history.
- Each role is provided access only to permitted functionalities.

4.2 Product & Inventory Management System

This module is the core of the SmartGrocery CLI system and allows administrators to manage grocery products and stock levels efficiently. All inventory data is maintained offline using file-based storage.

● Inventory Management Features

- Add new grocery products with name, category, price, and stock quantity.
- Update existing product details such as price and available stock.
- Delete products that are discontinued or incorrectly added.
- Categorize items into groups such as fruits, vegetables, dairy, and snacks.
- View the complete inventory in a structured CLI table format.

● Low-Stock Alert System

- The system automatically identifies products with stock below a predefined threshold.
- Low-stock warnings are displayed to alert administrators about refill requirements.

4.3 User Purchase & Cart Management

This module allows customers to manage their shopping activities efficiently. Each user is associated with a personal cart and purchase history.

● Cart Management Features

- Add products to the cart with selected quantities.
- Remove items or modify quantities in the cart.

- View cart contents and calculate the total cost dynamically.

● Checkout Process

- Generate a bill during checkout.
- Validate product availability before finalizing purchases.
- Automatically update inventory stock after a successful purchase.
- Save purchase details to the user's purchase history file.

4.4 Offline Recommendation Engine

The recommendation engine provides intelligent product suggestions based on user purchase behavior. All recommendations are generated offline using locally stored data.

● Recommendation Features

- Suggest products based on previous purchase history.
- Recommend items from frequently purchased categories.
- Offer budget-friendly alternatives within the same product category.

This feature enhances user experience without requiring internet access or external APIs.

4.5 Analytics & Reporting System

The analytics module provides meaningful insights into system usage and purchasing patterns. These reports assist both administrators and users in understanding trends and making informed decisions.

● System-Wide Analytics

- Identify the most frequently purchased products.

- Generate category-wise purchase statistics.
- Display low-stock product reports.

● User-Specific Reports

- Show total spending per user.
- Display recent purchase summaries.
- Provide simple weekly or monthly spending reports.

4.6 File-Based Data Management System

SmartGrocery CLI uses a file-based storage system to ensure data persistence and offline accessibility. All system data is stored in structured text files.

● Data Storage Features

- Store user information in users.txt.
- Maintain product and inventory data in products.txt.
- Record purchase history in purchases.txt.
- Save cart and transactional data as required.

● Data Integrity

- Files are read and written using controlled access methods.
- Data is updated consistently after each operation to avoid inconsistencies.

4.7 Command-Line Interface (CLI) Interaction

The system operates entirely through a terminal-based interface, complying with SPL-1 headless application requirements.

● CLI Capabilities

- Display structured menus for admin and customer roles.

- Provide guided input prompts for user actions.
- Show system messages, alerts, and reports clearly.

5. Tools & Technologies

5.1 Programming Language

We used Java as the primary programming language because it supports strong OOP architecture, cross-platform CLI development, and built-in file handling mechanisms essential for offline storage.

5.2 Development Environment

- VS Code — used for writing and running Java code with extensions for debugging and formatting
- Java Compiler (JDK) — to compile and execute code

5.3 Version Control

- Git for version tracking
- GitHub as the central remote repository to collaborate and maintain project updates
- This version control workflow ensures smooth teamwork, code reviews, and revision management.

6. Application Domain

Smart Grocery CLI belongs to the Retail Management and Inventory Management domain.

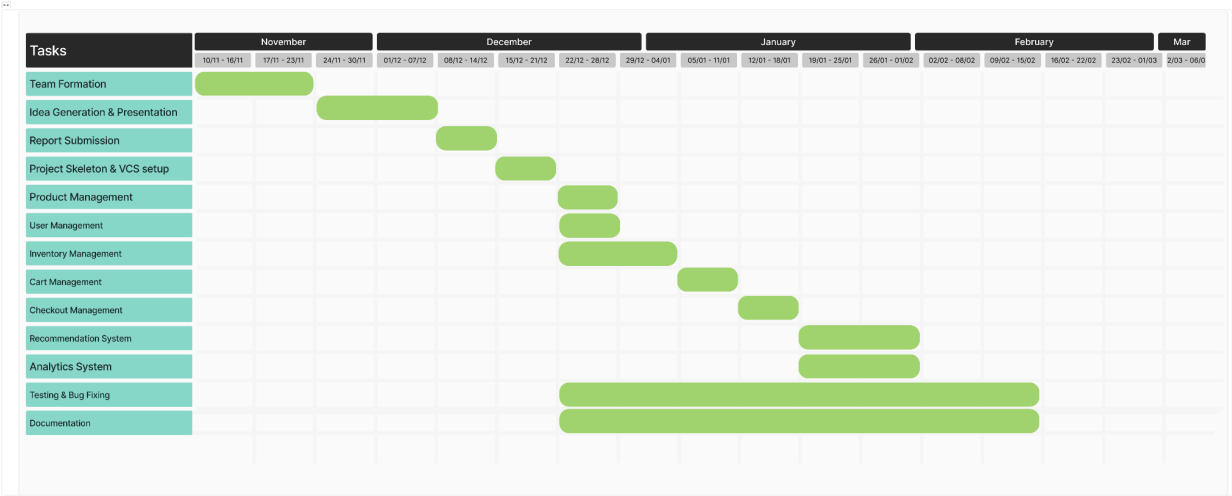
It is specifically designed for:

- Small grocery shops
- Local retail stores
- Offline inventory systems
- Small businesses without internet-based infrastructure

The system provides a lightweight digital solution for managing inventory and customer purchases efficiently.

7. Proposed Project Timeline

Timeline Diagram :



Planning and Preparation (Week 1-6)

During the first six weeks of the project, our focus was on building a strong foundation. The team was formed and roles were discussed to ensure smooth collaboration. We generated project ideas, analyzed real-world problems, and finalized the project concept through discussions and feedback. After idea finalization, we prepared and delivered the project proposal presentation. Finally, we completed the initial report submission, designed the basic project skeleton, and set up version control using Git and GitHub to manage code and track progress effectively.

Product Management Module (Week 6–7)

In this phase, the product-related functionalities are implemented. The Product and Inventory classes are developed to handle grocery item information such as name, category, price, and stock quantity. Features for adding, updating, deleting, and viewing products are implemented. File-based

storage mechanisms are integrated to persist product data. Initial validation and error-handling logic are also introduced to ensure data consistency.

User Management Module (Week 7–8)

This phase focuses on implementing user-related functionalities. The User and AuthenticationManager classes are developed to support user registration and login. Role-based access control is introduced to differentiate between Admin and Customer users. Purchase history tracking is initialized for each user. User data is stored and retrieved using file-based storage, and authentication workflows are tested through the CLI interface.

Inventory Management Module (Week 6–8)

During this phase, inventory control logic is enhanced. Stock tracking and low-stock alert mechanisms are implemented to notify administrators when product quantities fall below a defined threshold. Inventory update operations are integrated with product management and checkout processes. This phase ensures that inventory data remains accurate and synchronized after each purchase.

Cart & Checkout System (Week 8–10)

This phase implements the core purchasing workflow. The Cart class is developed to manage selected products and quantities for each user. The Checkout class is implemented to validate product availability, calculate total costs, update inventory stock, and generate purchase records. Purchase data is saved in files, and user purchase histories are updated accordingly. The complete purchase flow is tested to ensure reliability and correctness.

Recommendation & Analytics Module (Week 11–13)

In this phase, intelligent features are added to enhance user experience. The RecommendationEngine analyzes user purchase history to suggest relevant products and budget-friendly alternatives. The Analytics module is implemented to generate reports such as most bought items, category-wise statistics, user spending summaries, and low-stock reports. All computations are performed offline using locally stored data.

Documentation, Testing & Bug Fixing (Week 6–14)

This phase runs continuously throughout the project timeline. Tests are performed after each module implementation to verify functionality and catch defects early. Bugs and performance issues are identified and resolved iteratively. Project documentation, including UML diagrams, feature descriptions, and user guides, is prepared and refined. Version control practices are maintained using Git to track progress and individual contributions.

8. Contribution

Ayman Rahman Bhuiyan (230042140):

- Product Module:
  Built the Product model to represent grocery items in our system. designed it to track essential product information including categories, companies, pricing details, and stock levels.

  Also implemented product identification through unique IDs and integrated unit type support to handle different measurement systems (kg, liters, pieces, etc.).

- Purchase Module:
  Developed the Purchase module to record and manage customer transactions.

  Implemented the ability to store order compositions using product-quantity mappings, track when each purchase was made using timestamps, and calculate total order amounts. This module serves as the backbone for all purchase history and transaction record persistence throughout the system.

- Analytics Module:
  Created the Analytics engine to extract meaningful business insights from sales data.

  Implemented sales analytics features that help identify top-performing and underperforming products.
  Built the ability to generate comprehensive sales summaries including total orders, products sold, revenue earned, and average order value.

Developed an intelligent low stock detection system that goes beyond simple threshold-based alerts. Instead of just checking if stock is below a fixed number

Engineered a sales velocity algorithm that automatically calculates how many days of stock remain based on recent sales patterns. This helps prevent stockouts while avoiding over-purchasing.

Designed reporting capabilities to generate daily and weekly sales reports, track product sales history with daily breakdowns, analyze revenue by category, and provide insights into individual user spending patterns.

Also created specialized data structures (SalesSummary, SalesReport, LowStockAlert, ProductSalesHistory) to organize and present this data effectively.

Leveraged Java Streams for efficient data processing, enabling quick sorting, filtering, and aggregation of large datasets while maintaining clean, readable code.

Arafat Hosen (230042151) :

- Designed and implemented the Cart system, enabling users to add, remove, and manage products efficiently.
- Developed the Inventory system, handling product storage, updates, and stock management.
- Implemented features for tracking product availability and maintaining accurate inventory records.
- Integrated Cart and Inventory functionalities to ensure real-time updates and consistency.
- Improved system logic for handling edge cases like out-of-stock items.
- Contributed to organizing and structuring the overall project workflow.
- Prepared and maintained comprehensive project documentation.
- Assisted in creating and updating reports, diagrams, and system descriptions.
- Ensured code clarity and documentation for better team collaboration and future maintenance.

Md. Jahirul Islam Shifat (230042156) :

- Initialized and structured the entire project, setting up the repository and organizing the codebase for scalable development.
- Designed and implemented the **User Management system**, including secure authentication using salted SHA-256 hashing and role-based access control (Admin and Customer).
- Developed authorization logic to ensure proper access distribution between different user roles.
- Built the **Checkout Management system**, handling the complete flow from cart processing to purchase confirmation and receipt generation.
- Implemented the **Recommendation System**, including features such as best sellers, budget-friendly suggestions, and history-based recommendations.
- Led the integration and combination of all modules, ensuring smooth interaction between authentication, inventory, shopping, and analytics components.
- Developed the **Command-Line Interface (CLI)** for both Admin and Customer, focusing on usability, navigation, and structured menu design.
- Designed and implemented the **File Management system**, handling persistent storage for users, inventory, transactions, receipts, and logs using structured text files.
- Worked on improving data flow and consistency across modules, ensuring reliable read/write operations.
- Contributed to overall system architecture, modularization, and maintainability of the project.
- Collaborated in debugging, testing, and refining features to ensure a stable and efficient system.

All team members contributed to testing, debugging, and documentation.

9. Future Work

Although the system is functional, several improvements can be implemented in the future.

Possible Improvements :

- Graphical User Interface (GUI)
- Database integration (MySQL or PostgreSQL)
- Online cloud-based system
- Mobile application integration
- Barcode scanning support
- Advanced recommendation algorithms
- Real-time sales analytics dashboard

These improvements would make the system more scalable and suitable for larger retail businesses.

10. Source Repository

Smart Grocery CLI - Github Repository